

Attacchi Web

NECKER

2 Dicembre 2025

Contents

1	Document Object Model (DOM)	2
1.1	Oggetti Fondamentali del DOM	2
1.2	Attributi Rilevanti del DOM e HTTP	2
2	HTTP Cookies	2
2.1	Meccanismo di Funzionamento	2
2.2	Tipologie di Cookie	2
2.3	Attributi dei Cookie	3
2.4	Accesso JavaScript ai Cookie	3
3	Same-Origin Policy (SOP)	3
3.1	Same-Origin Policy per il DOM (DOM SOP)	3
3.1.1	Criteri di Corrispondenza dell'Origine	3
3.1.2	Esempio di Corrispondenza Origini	3
3.1.3	Rilassamento della SOP (<code>document.domain</code>)	4
3.2	Same-Origin Policy per i Cookie	4
3.3	SOP e AJAX	4
4	Cookie di Autenticazione, Script Malevoli e CSRF	5
4.1	Session ID e Furto di Cookie	5
4.1.1	Rischi di Sicurezza (Cookie Theft)	5
4.1.2	Protezione dei Cookie	5
4.2	Cross-Site Request Forgery (CSRF)	5
4.2.1	Vettori di Attacco CSRF	5
4.2.2	Mitigazione CSRF	5
5	Cross-Site Scripting (XSS)	6
5.0.1	Impatto Potenziale di XSS	6
5.1	Tipi di Attacchi XSS	6
5.1.1	XSS Stored (Persistente)	6
5.1.2	XSS Reflected (Non Persistente)	6
5.1.3	XSS DOM-based	6
5.2	Mitigazione XSS e Sanitizzazione dell'Input	7
6	SQL Injection	7
6.1	Meccanismo di Attacco	7
6.2	Mitigazione di SQL Injection	7

1 Document Object Model (DOM)

Il **Document Object Model (DOM)** è la rappresentazione interna gerarchica di un documento HTML o XML. Serve come **API (Application Programming Interface)** che consente ai linguaggi di scripting (principalmente JavaScript) di accedere e manipolare la struttura, lo stile e il contenuto del documento (viene creato dall'**HTML** e rimane nella mem del browser).

1.1 Oggetti Fondamentali del DOM

- **Window Object:** Rappresenta la finestra del browser (o un frame) in cui è visualizzato il documento. È l'oggetto radice.
- **Document Object:** È una proprietà dell'oggetto `window` (`window.document`). Costituisce la radice della struttura gerarchica che rappresenta il contenuto del documento HTML.
- **Location Object:** È una proprietà dell'oggetto `window` (`window.location`) e contiene i componenti dell'URL del documento associato.

1.2 Attributi Rilevanti del DOM e HTTP

Table 1: Attributi DOM e Correlati HTTP

Attributo	Note
Location (HTTP header)	Inviato dal server per il reindirizzamento URL.
Domain (HTTP cookie attribute)	Consente al server di origine di aumentare lo scopo del cookie.
<code>document.domain</code>	Nome host da cui è stato caricato il documento.
<code>window.location.href</code>	URL del documento richiesto.
<code>document.URL</code> (sola lettura)	URL del documento caricato.

2 HTTP Cookies

HTTP è un protocollo **stateless** (senza stato). Gli **HTTP Cookies** sono un meccanismo di **work-around** per mantenere lo stato (es. preferenze linguistiche, carrelli della spesa) tra richieste successive.

2.1 Meccanismo di Funzionamento

1. **Impostazione:** Il server invia uno o più stringhe di dati di dimensioni limitate (il cookie) al client (browser) tramite l'header di risposta **Set-Cookie**.
2. **Memorizzazione:** Il client memorizza il cookie.
3. **Restituzione:** Il client include il cookie nella successiva richiesta HTTP inviata allo **stesso sito o pagine** (a seconda dello scopo) utilizzando l'header di richiesta **Cookie**.

I cookie sono coppie "**nome=valore**" seguite da zero o più attributi.

2.2 Tipologie di Cookie

- **Session Cookies:** Cookie di breve durata, memorizzati nella memoria del browser e cancellati alla chiusura della finestra.
- **Persistent Cookies:** Hanno una vita estesa definita dagli attributi **Max-Age** o **Expires** e vengono conservati anche dopo la chiusura del browser.

2.3 Attributi dei Cookie

Gli attributi opzionali controllano la durata, lo scopo e le restrizioni di sicurezza di un cookie:

- **Max-Age=av secondi** o **Expires=data**: Definisce la durata massima del cookie (persistent cookie).
- **Domain=av**: Aumenta lo scopo del cookie a un suffisso di dominio. Se impostato su `myhost.com`, il cookie è disponibile per `myhost.com` e tutti i suoi sottodomini. Non può essere impostato per TLD (es. `.com`) per motivi di sicurezza.
- **Path=av**: Controlla a quali percorsi del file system (pagine) del server di origine un cookie viene restituito. Il valore predefinito è la directory della risorsa richiesta.
- **Secure**: Se presente, il client deve inviare il cookie **solo** su connessioni crittografate (HTTPS/TLS).
- **HttpOnly**: Se presente, l'accesso al cookie è consentito **solo** tramite HTTP e negato all'API DOM (es. JavaScript) per motivi di sicurezza (mitigazione XSS).

2.4 Accesso JavaScript ai Cookie

L'API DOM `document.cookie` consente di accedere (leggere e scrivere) a tutti i cookie per il documento corrente. Questo accesso è negato se l'attributo `HttpOnly` è impostato.

3 Same-Origin Policy (SOP)

La **Same-Origin Policy (SOP)** è un principio di controllo degli accessi e isolamento per impedire che un documento o script caricato da una fonte (**origine**) possa interferire con (accedere o manipolare) risorse provenienti da un'altra origine.

3.1 Same-Origin Policy per il DOM (DOM SOP)

Le regole fondamentali per i documenti HTML e gli script sono:

1. Un documento base ha un'origine definita dal URI che lo ha recuperato.
2. Gli script e le immagini ereditano l'origine del documento HTML che li ha caricati.
3. Gli script possono accedere al contenuto (e alle proprietà del DOM) la cui origine assegnata **corrisponde** alla propria.

3.1.1 Criteri di Corrispondenza dell'Origine

Due origini sono considerate **identiche** solo se hanno un **triplice di origine** (**scheme, host, port**) identico.

- **Scheme** (Schema): Protocollo (es. `http`, `https`).
- **Host** (Host): Nome host con nome di dominio completo (FQDN). I sottodomini sono origini distinte.
- **Port** (Porta): Porta esplicita o porta predefinita del protocollo (es. 80 per HTTP, 443 per HTTPS).

3.1.2 Esempio di Corrispondenza Origini

- **Corrispondenza**: `http://site.com/file1` e `http://site.com/file2` (stesso triplice).
- **Non Corrispondenza**:
 - `http://site.com` e `https://site.com` (*scheme* differente).
 - `http://site.com` e `http://sub.site.com` (*host* differente).
 - `http://site.com:80` e `http://site.com:8080` (*port* differente).

3.1.3 Rilassamento della SOP (`document.domain`)

I sottodomini che desiderano cooperare (es. `catalog.mystore.com` e `orders.mystore.com`) possono esplicitamente rilassare la SOP impostando `document.domain` al loro suffisso comune (es. `mystore.com`). Ciò rende i loro oggetti DOM accessibili tra loro, ma anche a **tutti** gli altri sottodomini di quel suffisso.

3.2 Same-Origin Policy per i Cookie

La politica per i cookie è **diversa** da quella DOM SOP.

- **Porta Esclusa:** Un cookie restituito a un host (server) è disponibile a **tutte le porte** su quell'host; la porta è esclusa dall'origine del cookie.
- **Ambito Regolabile:** Gli attributi `Domain` e `Path` consentono al server di **ampliare o limitare** l'ambito del cookie, rispettivamente, a un **dominio suffisso** o a un **percorso URI prefissato**.

Esempio:

- Se un cookie viene impostato dal server `prodotti.mionegoziot.com` con l'attributo `Domain=mionegoziot.com` il browser invierà tale cookie anche ai sottodomini come `pagamenti.mionegoziot.com` o `blog.mionegoziot.com` (ampliamento del dominio).
- Se un cookie viene impostato con l'attributo `Path=/carrello`, il browser lo invierà solo alle richieste che iniziano con quel percorso URI (ad esempio, `/carrello/checkout.html`), escludendo tutte le altre sezioni del sito come `/blog` (limitazione del percorso).

Il controllo sui cookie è basato sul percorso (`Path`), mentre l'accesso JavaScript agli HTTP cookie **non** è limitato dal percorso.

3.3 SOP e AJAX

La SOP si applica anche alle comunicazioni asincrone tra browser e server facilitate da **AJAX** (Asynchronous JavaScript and XML) e dall'oggetto `XMLHttpRequest`. Le richieste HTTP asincrone da parte degli script sono limitate all'utilizzo dello **stesso server di origine** che ha fornito il documento base in cui lo script è incorporato.

4 Cookie di Autenticazione, Script Malevoli e CSRF

4.1 Session ID e Furto di Cookie

I server utilizzano un **Session ID** (numero scelto casualmente) memorizzato in un cookie HTTP per mantenere lo stato di un'interazione (sessione). Una volta che un utente ha effettuato l'accesso, il Session ID diventa un **Authentication Cookie**, che autorizza le richieste successive senza richiedere la ri-autenticazione.

4.1.1 Rischi di Sicurezza (Cookie Theft)

Il furto dei cookie di autenticazione può portare all'hijacking della sessione e all'accesso non autorizzato a risorse sensibili. I principali vettori di furto includono:

- **JavaScript malevolo** (ad esempio, attacchi XSS): se il client esegue per errore uno script iniettato malevolo nella pagina legittima, può inviare il contenuto di `document.cookie` a un sito malevolo.
- **Proxy HTTP** o Man-in-the-Middle: se il cookie è inviato su connessioni non crittografate (HTTP chiaro).
- **Malware** client-side che accede alla memoria o al filesystem.

4.1.2 Protezione dei Cookie

Il server deve applicare misure di sicurezza sui cookie:

- **HttpOnly**: Previene l'accesso del cookie tramite JavaScript (mitigazione XSS).
- **Secure**: Forza l'invio del cookie solo su connessioni cifrate (HTTPS/TLS).
- **Integrità e Confidenzialità**: Il server dovrebbe **crittografare** e proteggere l'**integrità (MAC)** dei cookie che contengono dati sensibili prima di inviarli al client, per evitare modifiche lato client.

4.2 Cross-Site Request Forgery (CSRF)

La **Cross-Site Request Forgery (CSRF)** (o Session Riding) è un attacco che sfrutta la fiducia che un sito web ripone nel browser dell'utente. Se un utente è attualmente loggato (cioè possiede un Authentication Cookie), una richiesta HTTP **pre-autorizzata** generata dal browser della vittima (anche se avviata da un sito malevolo) viene eseguita dal server bersaglio (infatti dato che la fiducia è riposta sul browser, se navigando su un altro sito inoltra una richiesta alla banca fingendosi il sito ufficiale, l'attacco andrebbe a buon fine). L'attaccante non deve mai conoscere il contenuto del cookie.

4.2.1 Vettori di Attacco CSRF

L'attaccante induce la vittima a inviare una richiesta HTTP dannosa al sito bersaglio, sfruttando elementi HTML che innescano richieste automatiche:

- **Link** (`<a href="...">`): Richiede l'azione (click) della vittima.
- **Tag Immagine** (``): L'elemento più comune; il browser tenta automaticamente di caricare l'immagine, innescando una richiesta GET all'URL malevolo senza che la vittima se ne accorga (spesso con dimensioni 0×0 pixel).
- **Form Nascosti** con event handler (`onload`): Per generare richieste POST automaticamente.

4.2.2 Mitigazione CSRF

La difesa principale contro CSRF è l'uso di **Secret Validation Tokens (Anti-CSRF Tokens)**.

- All'inizio di una sessione, il server invia un token segreto unico al browser.
- Ogni richiesta HTTP successiva che altera lo stato sul server (POST, PUT, ecc.) deve includere questo token (così facendo una richiesta esterna con comportamento grave non è abilitata).
- Un attaccante CSRF, non avendo accesso a questo token (che non viene automaticamente inviato con i cookie), non può costruire la richiesta forgiata.

5 Cross-Site Scripting (XSS)

La **Cross-Site Scripting (XSS)** è una vasta classe di attacchi basati sull'iniezione di tag HTML malevoli o script in pagine web. Questo porta all'esecuzione di codice JavaScript non intenzionale sul browser della vittima, violando lo spirito della SOP.

5.0.1 Impatto Potenziale di XSS

L'esecuzione di script iniettati può portare a:

1. Accesso e **furto di Authentication Cookies** e token di sessione (il caso classico).
2. **Reindirizzamento** del browser a siti controllati dall'attaccante (phishing).
3. Accesso o modifica dei **dati DOM** e dei dati memorizzati nel browser per il sito corrente.
4. **Riscrivere il documento** visualizzato al client (`document.write()`).

5.1 Tipi di Attacchi XSS

5.1.1 XSS Stored (Persistente)

- Il codice malevolo viene **memorizzato in modo permanente** sul server (es. in un database o filesystem), come in un commento di un forum o un post di un blog, se non filtrato, verrà inserito nel database così come è.
- Quando qualsiasi visitatore carica la pagina contenente il contenuto memorizzato, lo script viene eseguito nel contesto di origine del sito legittimo poichè interpretato come codice.

5.1.2 XSS Reflected (Non Persistente)

- Lo script malevolo non viene memorizzato, ma è contenuto nell'URL della richiesta.
- Un sito vulnerabile **riflette** l'input non sanificato dell'utente (es. un parametro URL) nella pagina di risposta (es. una pagina di errore "File non trovato: *nomefile-iniettato*").
- Il browser della vittima esegue lo script riflesso. L'attaccante ingegnerizza la vittima a cliccare un link che contiene lo script, ad esempio invia via mail il link legittimo con attaccato lo script malevolo, una volta premuto il codice malevolo verrà eseguito.

5.1.3 XSS DOM-based

- L'iniezione e l'esecuzione avvengono interamente **lato client**.
- Il codice vulnerabile manipola l'ambiente DOM del browser in base a dati provenienti da fonti non attendibili (es. frammento URL, `document.location`), senza che vi sia un'interazione diretta con il server.

Esempio di XSS DOM-based

Questo attacco sfrutta la manipolazione del DOM **lato client** da parte di codice JavaScript che utilizza **dati non attendibili** (spesso dal frammento URL) come input.

- **Codice Vulnerabile (Lato Client):** Un codice JavaScript legittimo strutturato male che prende per esempio l'input dal frammento URL (`document.location.hash`) e lo inserisce direttamente nell'HTML della pagina, ad esempio tramite `document.write()`:

```
document.write("Benvenuto: " + document.location.hash.substring(1));
```

- **Vettore d'Attacco (URL Forgiato):** L'attaccante induce la vittima a caricare un URL che contiene il codice malevolo nel frammento, che **non** viene inviato al server:

```
http://sito.com/page.html#<script>alert('XSS')</script>
```

- **Esecuzione (Locale):**

1. Il browser carica la pagina; il server **non vede** il frammento (`#...`).
2. Il JavaScript vulnerabile legge il frammento e lo inserisce nell'HTML come codice.
3. Lo script `<script>alert('XSS')</script>` viene eseguito nel contesto di origine (`sito.com`).

Conclusione: Il server è bypassato; la vulnerabilità risiede esclusivamente nella logica del codice JavaScript che si fida dell'input locale.

5.2 Mitigazione XSS e Sanitizzazione dell'Input

Il principio P15 (**DATA-TYPE-VERIFICATION**) è fondamentale. La difesa si basa sulla **Sanitizzazione dell'Input**, il processo di rimozione di elementi potenzialmente malevoli dai dati inseriti dall'utente.

- **Output Escaping (Filter Evasion):** Sostituire caratteri speciali come `<` e `>` con le loro entità HTML (es. `<`; e `>`). Il browser li visualizza come testo letterale invece di interpretarli come sintassi di markup.
- **Canonicalizzazione:** Mappare l'input a un'unica codifica standard per neutralizzare tattiche di evasione come la **codifica alternativa** (es. usare `<` invece di `<`).
- **Whitelisting (Validazione Positiva):** L'approccio preferito. Consentire solo input "noto-buono" e strutture attese (es. se si attende un nome utente, rifiutare la sintassi `<script>`).

6 SQL Injection

La **SQL Injection** sfrutta la pratica di costruire dinamicamente query SQL lato server utilizzando l'input non sanificato dell'utente (ad esempio, da form, cookie o variabili).

6.1 Meccanismo di Attacco

L'attaccante inserisce dati che **alterano la struttura sintattica** della query SQL, aggiungendo comandi propri.

- **Sfruttare le Quote Singole:** Inserendo una quote singola (`'`) l'attaccante chiude prematuramente una stringa SQL aperta nel codice del server, introducendo comandi propri.
- **Commenti:** L'uso di `--` (in alcuni dialetti SQL) commenta la parte restante della query originale, eliminando i controlli di sicurezza (es. la verifica della password).
- **Condizioni Logiche:** Inserendo una condizione sempre vera (es. `' OR 1=1 --`), si forza l'esecuzione della query su tutti i record.
- **Separatori di Comando:** L'uso di un punto e virgola (`;`) può permettere l'esecuzione di una seconda istruzione SQL arbitraria (es. `DROP TABLE pswdtab --`).

6.2 Mitigazione di SQL Injection

La difesa si concentra sulla separazione netta tra il codice SQL e i dati di input.

- **Prepared Statements (Query Parametrizzate):** Il metodo di difesa primario. Il codice SQL viene inviato al database separatamente dai parametri di input. Il database distingue sempre il codice dai dati, assicurando che l'input sia trattato solo come valore letterale e non come comando.
- **Stored Procedures:** Simili alle Prepared Statements, memorizzano la query nel database e accettano solo parametri, impedendo l'iniezione.
- **Input Validation (Whitelisting):** Assicurarsi che l'input corrisponda al tipo e alla struttura attesi (es. verificare che un campo numerico contenga solo cifre).
- **Escaping:** Sostituire i caratteri speciali come la quote singola con la loro rappresentazione letterale (es. `'` o `\'`). Questo è un metodo reattivo e non sufficiente da solo.